HYBRID TELECOM STREAM PROCESSING FRAMEWORK

MPI-Based Multi-DES Communication for Distributed Telecom Packet Processing

Summer Internship Project Report

Prepared by

Pathi Jahnavi

Malaviya National Institute of Technology (MNIT), Jaipur

Katta Sreeja Meharani

G. Narayanamma Institute of Technology and Science (GNITS), Hyderabad

Project Guidance

Dr. V. C. V. Rao

MPPLAB, Centre for Development of Advanced Computing (C-DAC), Pune

May – July 2026

Contents

1. Objective
2. Background and Introduction
    - 2.1 Centre for Development of Advanced Computing (C-DAC)
    - 2.2 MPPLAB and the Present Project's Place Within It
    - 2.3 Telecom Networks, Data Extraction, and Data Extraction Servers
    - 2.4 Motivation
    - 2.5 Problem Statement and Need for the Project
3. Description of Present Work
    - 3.1 Overall Scope of the Delivered System
    - 3.2 Layer Responsibilities
    - 3.3 Repository Organisation
    - 3.4 Development Methodology
    - 3.5 Scope Boundaries
    - 3.6 Coding Standards and Documentation Practice
4. Computing Systems, Programming Environment and Tools Used
5. Methodology and Implementation
    - 5.1 System Overview and Architecture
    - 5.2 Communication Layer — MPI Multi-DES Design
    - 5.3 Multi-DES Communication Flow and Packet Lifecycle
    - 5.4 Processing Layer (POSIX Threads)
    - 5.5 Storage Layer (Berkeley DB)
    - 5.6 Graph Construction and Monitoring Layers
    - 5.7 Design Principles and Software Engineering Practices
6. Project Evolution
    - 6.1 Phase Summary
    - 6.2 Engineering Rationale and Lessons Learned per Phase
7. Results and Discussion
    - 7.1 Throughput Under Increasing DES Count and Load
    - 7.2 Latency
    - 7.3 Multi-DES Correctness and Queue Behaviour
    - 7.4 Graph Construction Results
    - 7.5 Scalability and Current Limitations
8. Testing and Validation
    - 8.1 Testing Methodology and Test Configurations
    - 8.2 System Component Validation

8.3 Communication, Processing, and Storage Testing
8.4 Integration and Scalability Testing
8.5 Testing Limitations
8.6 Software Engineering Value of the Testing Discipline
9. Software Developed
10. Conclusions and Future Directions
10.1 Conclusions
10.2 Future Directions
10.3 Reflections on Software Engineering Practice

1. Objective

The objective of this internship project was to design, implement, and evaluate a Hybrid Telecom Stream Processing Framework capable of ingesting telecom traffic from multiple, concurrently active Data Extraction Servers (DES), using the Message Passing Interface (MPI) as the primary communication mechanism between the DES clients and a central processing node.

The project's central focus was the MPI-based communication layer: establishing a reliable client-server architecture in which an arbitrary number of DES processes can transmit telecom packets concurrently to a single server without modification to the server's core receive logic, and validating this multi-DES communication pattern under increasing client counts and traffic volumes.

In support of this communication-centric goal, the project additionally integrated a small set of complementary layers — concurrent packet processing using POSIX threads, persistent storage using Berkeley DB, communication-graph construction, and runtime monitoring — so that packets received over MPI could be fully processed, stored, and observed end to end. These supporting layers are fully implemented, independently tested parts of the framework rather than incidental extras; they are described in less technical depth than the communication layer in this report because they were not the project's primary focus, not because they are peripheral to it.

The project followed an incremental development methodology in which POSIX Threads, TCP socket communication, socket-thread integration, and MPI-based communication were each independently designed, implemented, tested, and documented prior to their combination into the final hybrid framework.

Real telecom networks extract traffic from a large and, in general, dynamically changing number of geographically distributed exchanges and towers. A communication layer intended to support such an environment must therefore be able to accept traffic from a growing number of independent sources without requiring the central processing infrastructure to be redesigned each time a new source is added. Demonstrating this property under a controlled, multi-DES test environment, rather than assuming it, was accordingly set as the specific, measurable objective against which the communication layer described in this report was designed and evaluated.

2. Background and Introduction

This chapter situates the Hybrid Telecom Stream Processing Framework within the organisation at which it was developed and the research programme it supports, before the project's own objective, scope, and design are discussed in the chapters that follow.

2.1 Centre for Development of Advanced Computing (C-DAC)

The Centre for Development of Advanced Computing (C-DAC) is a premier research and development organisation under the Ministry of Electronics and Information Technology (MeitY), Government of India, engaged in the design, development, and deployment of advanced computing, communication, and information technology solutions. Since its inception, C-DAC has played a central role in the indigenous

development of supercomputing capability in India, most notably through the PARAM series of supercomputers, and has since diversified into high performance computing, cyber security, health informatics, professional electronics, and language computing.

C-DAC Pune, where this internship was carried out, houses several specialised laboratories focused on distinct areas of advanced computing research. Among these is the Multi-Petaflop Parallel Laboratory (MPPLAB), within which this project was undertaken. C-DAC's laboratories routinely host student interns from academic institutions across India, providing exposure to real research and development workflows, mentorship from experienced scientists and engineers, and the opportunity to contribute to ongoing national-scale computing initiatives.

2.2 MPPLAB and the Present Project's Place Within It

MPPLAB is a research group at C-DAC Pune whose broader objective is the development of parallel and distributed computing techniques and platforms to support mathematically oriented, high-productivity computing across a range of application domains. One of the application domains pursued within this programme is tele-traffic engineering, in which network traffic arising from telecommunication systems is analysed, optimised, and routed using parallel algorithms. Realising this capability requires an interface between telecom infrastructure and the underlying computing platform, so that traffic data can be extracted from the network, supplied to the relevant algorithms, and the resulting output subsequently applied back to the network.

The Hybrid Telecom Stream Processing Framework developed during this internship addresses precisely this class of problem, from the communication and data-ingestion end: it provides the MPI-based communication layer, together with supporting concurrent processing, storage, graph construction, and monitoring capabilities, required to reliably ingest telecom packet traffic from multiple, concurrently active Data Extraction Servers and make it available, in a structured and analysable form, for downstream tele-traffic engineering work. The framework does not itself perform tele-traffic optimisation; it is best understood as an infrastructural building block, demonstrating in a self-contained and independently testable form the parallel and distributed communication techniques such downstream work depends on.

2.3 Telecom Networks, Data Extraction, and Data Extraction Servers

Telecom networks generate continuous, high-velocity streams of call and traffic data across a large number of geographically distributed exchanges and towers. Telecom data extraction refers to the process by which this operational data is drawn out of network elements, formatted into a structured representation, and made available to downstream systems. In a production telecom environment, extraction typically occurs at multiple points simultaneously, since traffic is distributed across numerous exchanges and base stations rather than concentrated at a single location; a system intended to support such extraction at scale must therefore ingest data from multiple, concurrently active sources without becoming a bottleneck relative to the rate at which the network generates traffic.

Within the present framework, this multi-source extraction scenario is modelled through the Data Extraction Server (DES): a process that generates and transmits telecom packets to the central processing server. Each DES operates as an independent MPI client, generating packets with structurally valid source

and destination tower identifiers, traffic types, timestamps, and payloads, and transmitting them to the server using MPI point-to-point communication. The framework places no fixed upper bound on the number of concurrently active DES processes, which is precisely the property exercised by the multi-DES communication layer described in Chapter 5 and evaluated in Chapter 7.

2.4 Motivation

The motivation for this project arises from a practical observation within the MPPLAB research programme: while considerable effort has been directed towards the mathematical modelling of telecom traffic, comparatively less infrastructure existed for reliably extracting, transporting, processing, and storing telecom packet data at the volumes and rates that realistic deployments would demand. A mathematically sound traffic model is of limited practical value if the underlying data pipeline feeding it cannot ingest data from multiple sources concurrently, cannot process it fast enough to keep pace with generation, or cannot persist it durably for subsequent analysis. This project was motivated by the need to build, and rigorously validate, precisely such a data pipeline using well-established parallel and distributed computing techniques, with the communication layer connecting multiple Data Extraction Servers to a central processing node treated as the primary engineering challenge.

2.5 Problem Statement and Need for the Project

The problem addressed by this project may be stated as follows: given telecom packet traffic generated concurrently by an arbitrary number of Data Extraction Servers, design and implement a communication layer that can reliably transport all generated packets to a central processing node without loss, together with the supporting processing, storage, graph-construction, and monitoring capabilities needed to act on that traffic once received. The framework must further be modular, such that the communication layer and each supporting layer can be independently developed, tested, and reasoned about, while remaining composable into a single integrated system.

Without a reliable, concurrent, and scalable communication and data-ingestion pipeline, any downstream traffic engineering, analytics, or visualisation capability built on telecom data would inherit the reliability and performance limitations of that pipeline, regardless of how sophisticated the downstream analysis might be. A pipeline built without regard for correctness under concurrency — for instance, one that could not accept traffic from more than one source at a time — would fail to reflect the distributed nature of real telecom data extraction. The incremental, modular approach adopted in this project, in which POSIX Threads, TCP sockets, socket-thread integration, and MPI communication were each independently implemented, tested, and validated before integration, directly addresses this need by establishing the correctness of the communication layer, and of each supporting mechanism, prior to their combination.

3. Description of Present Work

This chapter describes, at a level of detail sufficient to orient the reader before the technical discussion in Chapter 5, the overall scope of the work delivered during the internship, the responsibilities of the framework's constituent layers, and the repository in which the work is organised.

3.1 Overall Scope of the Delivered System

The work delivered under this internship is a single, integrated software artefact, the Hybrid Telecom Stream Processing Framework, together with the complete set of intermediate implementations from which it was built. The final framework accepts telecom packets from multiple Data Extraction Servers over MPI, buffers and processes them concurrently using a pool of POSIX worker threads, persists processed packets to a Berkeley DB database, constructs a communication graph from the observed traffic, and continuously reports monitoring statistics describing the internal state of every layer. The system was implemented entirely in the C programming language, targeting the Ubuntu Linux operating system, and built using GNU Make.

3.2 Layer Responsibilities

The framework is organised into five layers. The Communication Layer, the primary focus of this project, is responsible for establishing MPI-based connections between DES clients and the central server, and for the reliable serialisation, transmission, and reception of telecom packets across process boundaries, supporting an arbitrary number of concurrently active DES clients. The remaining four layers are supporting layers built around this communication core, each with a well-defined responsibility: the Processing Layer buffers and concurrently processes incoming packets using a POSIX worker thread pool; the Storage Layer persists every processed packet to Berkeley DB; the Graph Construction Layer models communication relationships between telecom endpoints as a weighted graph; and the Monitoring Layer aggregates and reports runtime statistics from every other layer. Each of these layers was implemented and independently tested in its own right, as described in Sections 5.4 to 5.6, even though the communication layer remained the project's central focus.

3.3 Repository Organisation

The project repository, telecom-stream-processing-framework, preserves the complete development history of the project rather than containing only the final integrated system, as summarised in Table 3.1.

Directory	Description
src/	Source code for all project phases and the final hybrid implementation.
docs/	Architecture, design, implementation, testing, and workflow documentation, including a submissions/ subdirectory holding the final thesis report, presentation slides, and associated PDF deliverables.
diagrams/	System architecture and project evolution diagrams.
tests_and_reports/	Testing artefacts, validation reports, performance reports, and phase-wise documentation.
logs/	Development and execution logs generated during implementation and testing.

Table 3.1: Repository Directory Organisation

Within src/, the hybrid/ subdirectory containing the final integrated framework is itself organised per module, with dedicated subdirectories for communication (MPI), processing, storage, graph, and

monitoring code, alongside the standalone pthreads/, sockets/, integration/, and mpi/ implementations preserved from the earlier development phases.

Within docs/, a submissions/ subdirectory holds the final academic deliverables for the internship: the thesis report, the accompanying presentation slides, and PDF renderings of both, kept separate from the architecture and design documentation described elsewhere in docs/ so that graded submission artefacts are not mixed with working technical documentation.

3.4 Development Methodology

Development followed an incremental software engineering methodology, in which each module or phase was carried through a consistent cycle of design, implementation, testing, validation, and documentation before being considered complete or being integrated with other modules. This cycle was applied uniformly across all five development phases summarised in Chapter 6, and in particular to the MPI communication layer, which was designed, implemented, and independently validated against a single-client baseline before being extended and tested against multiple concurrent DES clients.

At every stage, a corresponding validation report was produced and preserved in tests_and_reports/, so that the correctness of a given layer could be verified in isolation before it was relied upon by the layer built on top of it. This discipline was particularly important for the communication layer: because every other layer in the framework ultimately depends on packets delivered correctly and completely by the MPI communication layer, establishing confidence in that layer first, before building the processing, storage, graph, and monitoring layers around it, was treated as the highest-priority activity of the internship.

3.5 Scope Boundaries

Two boundaries were deliberately placed on the scope of the delivered work. First, the traffic ingested by the framework is synthetic, produced by the traffic-generation module described in Section 5.3, rather than drawn from a live telecom network feed; this was a deliberate simplification that allowed the communication and processing layers to be evaluated under controlled, repeatable conditions. Second, all testing was performed with DES clients and the server co-located on a single physical machine; genuinely distributed, multi-node deployment is identified as future work in Chapter 10, but the MPI communication layer was designed from the outset, described in Section 5.2, to require no code change to support such a deployment.

3.6 Coding Standards and Documentation Practice

A small set of coding conventions was applied consistently across all five source directories to keep the codebase legible to readers unfamiliar with any individual phase, an important property given that the repository is intended to remain usable by future MPPLAB interns and staff, as noted in Section 3.4. Every public function exposed by a module, listed in the API tables throughout Chapter 5 and consolidated in Section 9.4, is documented at its point of declaration with a short comment describing its purpose, parameters, and return behaviour. Function and variable names follow a consistent snake_case convention throughout the C source, matching the convention used by the MPI, POSIX Threads, and Berkeley DB libraries the framework builds upon, so that framework-specific and library code read uniformly. Error handling follows a single convention throughout: functions that can fail return a signed integer status code,

with a negative value indicating failure, rather than relying on global error state or silent failure, so that every call site in the communication, processing, storage, and graph layers can check for and propagate failure consistently.

Each source directory additionally contains its own README-style design note, consistent with the docs/ organisation summarised in Table 3.1, recording the responsibilities, key data structures, and known limitations of that phase at the time it was completed. This practice was applied from Phase 1 onward, described in Chapter 6, so that later phases, and in particular the final hybrid integration, could be developed with an accurate written record of each earlier phase's design decisions, rather than relying on institutional memory or a re-reading of the source code alone.

4. Computing Systems, Programming Environment and Tools Used

4.1 Programming Language and Operating System

The framework was implemented entirely in the C programming language and targets the Ubuntu Linux operating system, a natural fit given C's low-level control over memory layout, process and thread management, and its first-class support in MPI, POSIX Threads, and Berkeley DB toolchains, and given Ubuntu's status as a widely deployed Unix-like target for telecom infrastructure software.

4.1.1 Hardware Environment

All development, building, and testing described in this report was carried out on a single physical machine running Ubuntu Linux, with all DES client processes and the server process co-located on that machine and distinguished only by their MPI rank. This single-node configuration was sufficient to exercise and validate the multi-DES communication pattern that is the focus of this report, since MPI ranks communicate as independent processes regardless of whether they are co-located or distributed across a physical cluster; genuinely distributed, multi-node deployment is discussed as future work in Chapter 10.

4.2 MPI Environment

OpenMPI was selected as the MPI implementation on account of its wide adoption, mature tooling, and straightforward availability through the standard Ubuntu package repositories. All MPI programs, including the standalone MPI phase and the final hybrid framework, were compiled using the mpicc compiler wrapper and launched using the mpirun process manager.

4.3 Berkeley DB Environment

Berkeley DB was selected as the embedded storage engine on account of its lightweight footprint, well-documented C API, and suitability for high-throughput key-value storage without the operational overhead of a standalone database server process.

4.4 Tools and Version Control

Category	Tools / Technologies
Programming Language	C
Parallel Programming	POSIX Threads (Pthreads)
Distributed Communication	MPI (OpenMPI) — primary focus
Networking (early phase)	TCP Socket Programming
Database	Berkeley DB
Synchronisation	Mutexes, Condition Variables
Data Structures	Circular Queue, Graph
Build System	GNU Make
Operating System	Ubuntu Linux
Version Control	Git and GitHub

Table 4.1: Technology and Tool Stack

Git was used as the version control system, hosted on GitHub, preserving the complete phase-wise development history described in Chapter 6. A .gitignore file was maintained at the repository root to exclude build artefacts and generated database files from version control.

4.5 Development, Debugging, and Verification Tools

Because the framework combines multiple concurrency and communication mechanisms, correctness could not be established by inspection alone, and a small set of standard Linux development tools was used throughout implementation and testing to support it. GNU Debugger (gdb) was used to step through both the MPI communication paths and the POSIX-threaded processing paths during development, particularly when diagnosing the early-phase socket and thread-integration issues discussed in Chapter 6. Valgrind, and in particular its Helgrind tool for detecting data races and lock-ordering errors in multithreaded programs, was used against the standalone POSIX Threads phase and the processing layer of the hybrid framework to check for race conditions on the shared circular buffer beyond what functional testing alone could reveal. strace was used occasionally during the TCP socket phase to confirm the exact sequence of system calls made during connection setup and teardown, which proved useful in understanding the connection-management burden discussed in Section 5.2.1.

4.6 Compilation and Build Configuration

Each of the five source directories under src/ is built independently using its own GNU Make Makefile, rather than a single project-wide build system, consistent with the modular, phase-wise organisation of the repository described in Section 3.3. The MPI phases are compiled using the mpicc wrapper around gcc, which transparently adds the include paths and link flags required by the local OpenMPI installation, while the POSIX Threads and TCP socket phases are compiled with gcc directly, linking against the pthread and, where applicable, standard sockets libraries. The hybrid framework's Makefile additionally links against the

Berkeley DB client library required by the storage layer described in Section 5.5. Every Makefile defines a clean target that removes compiled object files and executables, allowing each phase to be rebuilt from a known state during testing.

5. Methodology and Implementation

This chapter presents the design and implementation of the Hybrid Telecom Stream Processing Framework. Consistent with the project's objective, the MPI-based Communication Layer and its multi-DES design are treated as the primary subject of this chapter (Sections 5.2 and 5.3) and are discussed in full technical detail. The Processing, Storage, Graph Construction, and Monitoring layers are supporting layers built around this communication core. They are described more briefly than the communication layer in Sections 5.4 to 5.6, since they were not the project's primary focus, but each is nonetheless a working, tested part of the framework, and the discussion below covers how they receive and act upon packets delivered by the communication layer.

5.1 System Overview and Architecture

The framework is organised as a layered pipeline in which telecom packets flow from generation at the Data Extraction Servers, through MPI-based distributed communication, concurrent processing, persistent storage, and graph construction, to final monitoring and reporting. Each layer exposes a narrow interface to its neighbours and encapsulates its internal implementation details, allowing each to be developed, tested, and reasoned about independently.

Figure 5.1: Overall Hybrid System Architecture

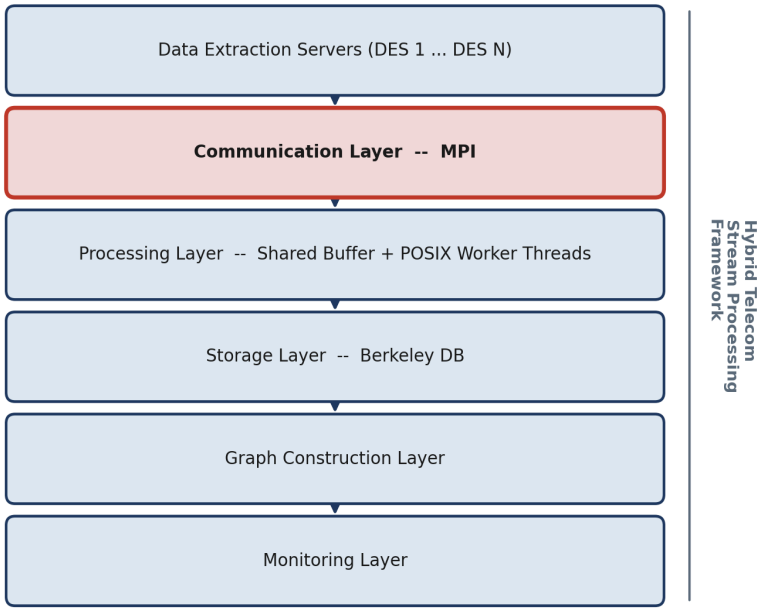

Figure 5.1: Overall Hybrid System Architecture

5.2 Communication Layer — MPI Multi-DES Design (Primary Focus)

The Communication Layer is implemented using the Message Passing Interface in a client-server configuration and is the architectural centrepiece of the framework. Each Data Extraction Server is realised as an independent MPI client process, which repeatedly generates telecom packets and transmits them to a single central server process using MPI point-to-point send operations. The server process, initialised via initialize_server(), receives packets from any connected client using receive_packet(), and hands each received packet to the processing layer for concurrent handling.

Because MPI clients operate as independent processes rather than threads sharing a server's address space, the communication layer naturally supports an arbitrary number of concurrently active Data Extraction Servers without any modification to the server's core receive logic. This property was deliberately exploited and is the single most important design decision in the framework: scaling the number of DES clients from one to three, as reported in Chapter 7, required no change whatsoever to the server implementation, only to the number of client processes launched via mpirun.

Figure 5.2: Multi-DES MPI Communication Architecture

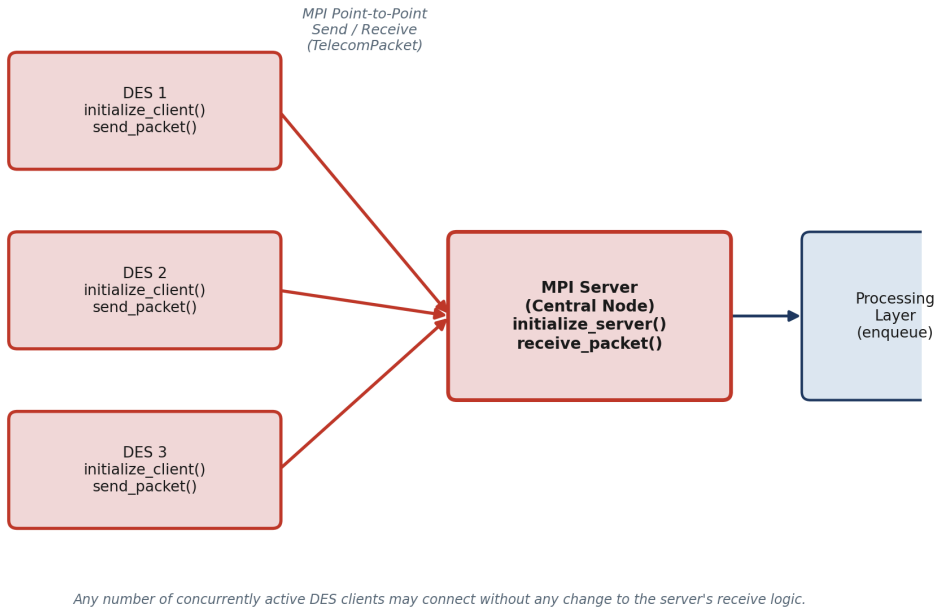

Figure 5.2: Multi-DES MPI Communication Architecture

The central data structure of the communication layer is the TelecomPacket, a fixed-layout structure comprising a source identifier, a destination identifier, a traffic type, a timestamp, and a payload field, serialised as a contiguous block for transmission via MPI and deserialised identically at the receiving end. Choosing a fixed-layout, contiguous structure, rather than a variable-length or pointer-based representation, was a deliberate design decision: it allows each packet to be transmitted using a single MPI send and a single MPI receive operation, without the additional complexity and overhead of a multi-message serialisation protocol — a decision that directly benefits the multi-DES scalability discussed in Chapter 7.

Function	Description
initialize_server()	Initialises the MPI server process (central node).
initialize_client()	Initialises an MPI client process, one per Data Extraction Server.
send_packet()	Transmits a single serialised TelecomPacket from a DES client to the server.
receive_packet()	Receives a TelecomPacket at the server from any connected DES client.

Table 5.1: Public API — Communication Layer

This MPI-based communication module was the fourth of the four independently developed communication and concurrency phases described in Chapter 6, superseding an earlier TCP socket-based implementation once MPI was judged better suited to the multi-client communication pattern required by multiple concurrently active DES clients; the socket-based implementation is nonetheless preserved in src/sockets for reference.

5.2.1 Why MPI Was Chosen Over TCP Sockets for Multi-DES Communication

The decision to replace the phase-two TCP socket implementation with MPI, rather than extending the socket implementation directly, was driven specifically by the multi-DES requirement. A raw TCP socket server must explicitly accept() each incoming client connection, maintain a table of open file descriptors, and either fork a new handling process or spawn a new thread per client, or multiplex all open sockets through select()/poll()/epoll(); adding a new DES client therefore requires the server's connection-handling logic to correctly manage an additional, dynamically arriving file descriptor. MPI, by contrast, establishes the communicator across all participating processes at start-up time, through mpirun, so that receive_packet() can accept a message from any source without the server ever needing to manage connection state explicitly. Table 5.2 summarises the comparison as it applies specifically to the multi-DES scenario evaluated in this project.

Concern	TCP Sockets (Phase 2)	MPI (Phase 4, adopted)
Adding a new DES client	Server must accept() a new connection and track a new file descriptor	New client simply joins the communicator at mpirun launch time
Multi-client receive logic	Requires select()/poll() or one thread per connection	A single receive_packet() call accepts from any source
Message framing	Application must delimit message boundaries over a byte stream	MPI preserves message boundaries natively
Process model	Client and server are independent OS processes over a socket	Client and server are MPI ranks within one communicator
Suitability for this project	Validated and preserved, but not extended further	Adopted as the framework's primary communication layer

Table 5.2: TCP Sockets vs MPI for Multi-DES Communication

This comparison was established empirically, not merely theoretically: the TCP socket phase was implemented and validated first, and the observations recorded during that validation, specifically the

additional bookkeeping required each time a new client connection was introduced, directly motivated the subsequent MPI phase described in Chapter 6.

5.2.2 TelecomPacket Serialisation Format

Table 5.3 summarises the fixed-layout fields of the TelecomPacket structure transmitted in a single MPI send/receive pair between a DES client and the server.

Field	Purpose	Populated By
source	Identifier of the originating telecom tower	generate_source()
destination	Identifier of the destination telecom tower	generate_destination()
traffic_type	Category of telecom traffic (e.g. voice, data, SMS)	generate_traffic_type()
timestamp	Simulated time at which the packet was generated	generate_timestamp()
payload	Representative packet payload content	generate_payload()

Table 5.3: TelecomPacket Fields

Because every field has a fixed size known at compile time, the entire structure is transmitted as one contiguous block, so the size of every message exchanged over MPI is identical regardless of which DES client sent it or what traffic type it carries. This uniformity was a deliberate simplification that kept the communication layer's implementation independent of the number or identity of the connected DES clients.

5.2.3 Fairness and Ordering Across Multiple DES Clients

Because receive_packet() is implemented using MPI_ANY_SOURCE, as shown in Algorithm 5.2, the server does not enforce any fixed priority or round-robin schedule among connected DES clients; packets are received in whatever order the underlying MPI runtime delivers them, which in practice interleaves packets from all active clients rather than draining one client's traffic before servicing another. This was verified during multi-DES testing, described in Section 7.3, by confirming that the graph construction layer's recorded communication volume, which accumulates contributions from every DES client into a single shared graph, matched the combined total packets generated across all clients in every test configuration, rather than reflecting only one client's traffic.

5.3 Multi-DES Communication Flow and Packet Lifecycle

Figure 5.3 traces a single telecom packet from generation at a DES client through to its final appearance on the monitoring dashboard, illustrating how the MPI communication layer hands off each received packet to the supporting layers.

Figure 5.3: End-to-End Telecom Packet Lifecycle

Figure 5.3: End-to-End Telecom Packet Lifecycle

Each DES client independently and continuously executes a traffic-generation loop: generate_packet() invokes generate_source() and generate_destination(), which select source and destination tower identifiers from a bounded, shared pool; generate_traffic_type(), which selects a traffic category; generate_timestamp(); and generate_payload(). The resulting TelecomPacket is transmitted to the server via send_packet(). Because each DES client runs as an independent MPI process, all three DES clients used in testing generate and transmit traffic concurrently and asynchronously with respect to one another and to the server, which is precisely the multi-DES operating condition the communication layer was designed to support.

At the server, receive_packet() accepts packets from any connected client in the order MPI delivers them, without preference for a particular DES, and immediately enqueues each received packet into the shared circular buffer for downstream processing, described in Section 5.4.

5.3.1 DES Client and Server Control Flow

Algorithm 5.1 and Algorithm 5.2 summarise, respectively, the control flow executed independently by every DES client process and the control flow executed once by the server process, as implemented in src/hybrid/mpi.

```
Algorithm 5.1  DES Client Loop (executed independently by every DES)
1:  initialize_client()
2:  while traffic generation is active do
3:      pkt <- generate_packet()
4:      send_packet(pkt, server_rank)
5:  end while
Algorithm 5.2  Server Receive Loop (single server rank, any number of DES clients)
1:  initialize_server()
2:  buffer_init()
3:  spawn worker thread pool (Section 5.4)
4:  while server is active do
5:      pkt <- receive_packet(MPI_ANY_SOURCE)
6:      enqueue(pkt)
7:  end while
```

The server's loop, Algorithm 5.2, contains no reference to how many DES clients are currently active: receiving from MPI_ANY_SOURCE at line 5 is precisely the mechanism that allows the number of

concurrently connected DES clients to be increased, as in the scalability results of Chapter 7, by launching additional client ranks under mpirun, without altering a single line of the server implementation.

5.3.2 Validation on Receipt

Every packet received by the server, regardless of which DES client transmitted it, is passed to validate_packet() before being handed to the storage and graph layers, described in Sections 5.5 and 5.6. validate_packet() checks that the source and destination fields fall within the expected range of valid tower identifiers and that the traffic_type field is one of the recognised traffic categories, rejecting structurally malformed packets before they can propagate into the persistent store or the communication graph. This check is applied uniformly to packets from every DES client, so a malfunctioning or misconfigured client cannot corrupt the shared state built up from the traffic of the other, correctly functioning clients.

5.4 Processing Layer (POSIX Threads)

The Processing Layer decouples packet reception from packet processing through a shared circular buffer, following the producer-consumer pattern: the MPI receive path acts as the producer, inserting packets via enqueue(), while a fixed-size pool of POSIX worker threads acts as the consumers, each repeatedly removing packets via dequeue() and invoking process_telecom_packet(). Access to the buffer is protected by a mutex, with two condition variables (not_empty, not_full) used to avoid busy-waiting on either side. This design allows the rate at which the MPI communication layer receives multi-DES traffic and the rate at which it is processed to vary independently, with the buffer absorbing short-term bursts from multiple simultaneously active DES clients.

Figure 5.4: Producer-Consumer Shared Buffer and Worker Pool

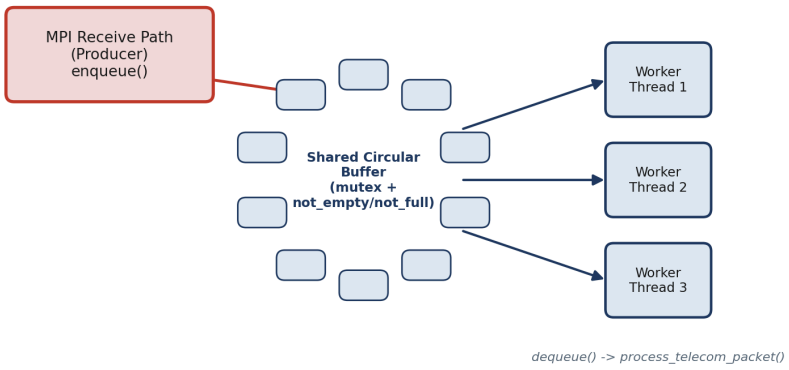

Figure 5.4: Producer-Consumer Shared Buffer and Worker Pool

The number of worker threads created at server start-up is a configurable parameter, allowing the degree of processing parallelism to be tuned independently of the number of connected DES clients; this separation of concerns, receiving multi-DES traffic on the one hand and processing it on the other, kept the addition of the processing layer from constraining the communication layer's design in any way.

5.5 Storage Layer (Berkeley DB)

The Storage Layer wraps the Berkeley DB C API behind four functions: db_initialize() opens the telecom.db database file; db_store_packet() serialises and stores a validated packet under a key derived from its identifying fields; db_get_packet() retrieves a previously stored packet; and db_close() flushes and closes the database. An embedded library was chosen, rather than a client-server database, specifically to avoid the additional latency that a networked database would introduce into the per-packet path fed by the MPI communication layer, given the sustained throughput requirement discussed in Chapter 7.

Every packet that reaches the storage layer has already been validated by validate_packet() and processed by process_telecom_packet(), so db_store_packet() performs no further correctness checking of its own; it is solely responsible for the mechanics of serialisation and persistence. This division of responsibility keeps the storage layer simple and keeps validation logic in exactly one place, the Processing Layer, regardless of how many DES clients contributed the packet being stored.

5.6 Graph Construction and Monitoring Layers

The Graph Construction Layer maintains vertices representing telecom towers and weighted, directed edges representing observed communications between them. initialize_graph() allocates the structure; add_vertex() idempotently inserts a tower; add_edge() creates or increments a weighted edge; print_graph_statistics() reports aggregate counts; and get_top_communication_link() identifies the single most active communicating pair. Because packets from every DES client pass through the same processing pipeline before reaching the graph layer, the resulting graph reflects the combined communication pattern of all connected DES clients, rather than any single client in isolation, which is what makes the multi-DES graph results in Section 7.3 meaningful.

Figure 5.5 illustrates this conceptually for a small set of representative telecom endpoints, modelled on city-level exchanges, with edge weights denoting observed communication volume between endpoint pairs, and shows, beneath the graph, the underlying worker processes at a single Data Extraction Server partition that generate the traffic contributing to those edges — the same binding of concurrent workers to packet partitions used throughout the traffic-generation and processing layers described in Sections 5.3 and 5.4.

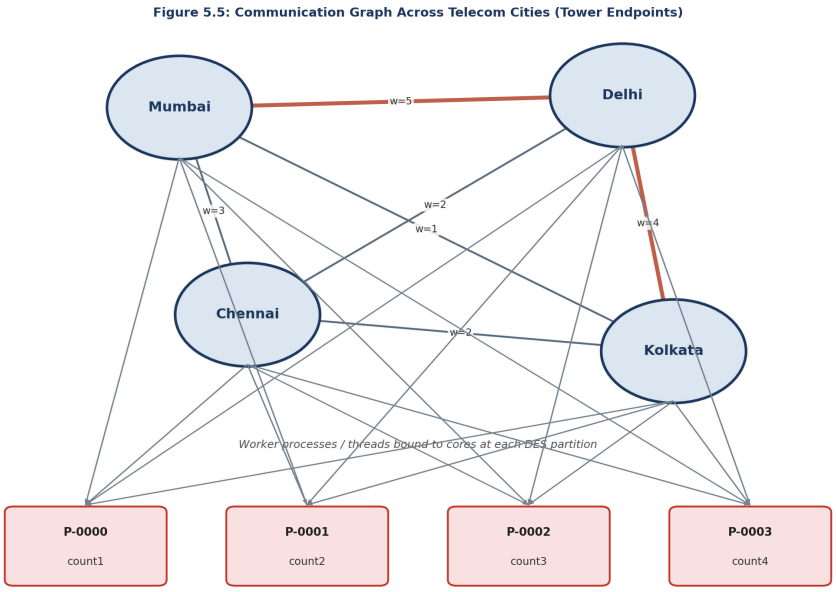

Figure 5.5: Communication Graph Across Telecom Endpoints

The Monitoring Layer maintains no independent state of its own, instead querying the counters maintained by every other layer on demand: monitoring_initialize() records a start timestamp, monitoring_get_statistics() aggregates current counters and computes throughput, and monitoring_print_dashboard() renders this aggregated state for human-readable display, used throughout development and validation to confirm that packets received over MPI from every DES client were, in every case, fully accounted for downstream. A representative excerpt of the dashboard's console output is shown below.

```
===== Hybrid Telecom Stream Processing Framework: Monitoring Dashboard =====
  DES clients connected       : 3
  Packets received (MPI)      : 3000
  Packets processed           : 3000
  Packets enqueued/dequeued   : 3000 / 3000
  Packets stored (Berkeley DB): 3000
  Graph vertices / edges      : 10 / 100
  Elapsed time (s)            : 0.0616
  Throughput (pkts/s)         : 48685.29
```

5.7 Design Principles and Software Engineering Practices

Beyond the specific technical design decisions described in the preceding sections, the methodology followed throughout Chapter 5 was governed by a small number of software engineering principles applied consistently across the communication layer and its supporting layers.

- Separation of concerns: each layer, communication, processing, storage, graph construction, and monitoring, owns a single, clearly bounded responsibility and exposes a narrow public API, listed in the tables throughout this chapter, through which other layers interact with it; no layer reaches into another layer's internal state directly.
- Narrow, stable interfaces: the communication layer's public API (Table 5.1) did not change when the number of supported DES clients was scaled from one to three, described in Section 5.2, precisely

because initialize_server(), receive_packet(), and the remaining functions were designed around the number of clients as a runtime, rather than compile-time, concern.

- Defensive validation at layer boundaries: every packet is passed through validate_packet() immediately on receipt, before it is handed to the processing, storage, or graph layers, described in Section 5.3.2, so that a malformed or malicious packet from one DES client cannot corrupt state shared with the other layers or with other clients.
- Incremental validation before integration: as described further in Chapter 6, each concurrency and communication mechanism, POSIX Threads, TCP sockets, socket-thread integration, and MPI, was implemented and independently tested before being combined, so that a defect could be localised to a single, recently integrated mechanism rather than searched for across the whole system.
- Observability by design: the monitoring layer described in Section 5.6 was not added retroactively but was designed in from the outset specifically so that the internal consistency invariants reported in Chapter 8, packets received equal to packets processed, enqueued equal to dequeued, and so on, could be checked continuously during development rather than only at the end of a test run.

These principles are not specific to any one layer; they were applied most rigorously to the communication layer, consistent with its status as the primary focus of the project, but were carried through consistently to the supporting layers as well, so that the correctness of the whole system could be reasoned about compositionally rather than only by testing the fully integrated framework as a single unit.

6. Project Evolution

The final hybrid framework was not implemented directly; it was arrived at through a sequence of independently designed, implemented, tested, and documented phases, each introducing a distinct concurrency or communication capability before those capabilities were combined, as illustrated in Figure 6.1.

Figure 6.1: Phase-Wise Project Evolution

Figure 6.1: Phase-Wise Project Evolution

In the first phase, a standalone POSIX Threads implementation established and validated the shared circular buffer and worker thread pool in isolation from any networked communication. In the second phase, a TCP socket-based communication implementation was developed and validated independently. In the third phase, the socket-based communication layer was integrated with the threading infrastructure, validating that the two concurrency mechanisms could be composed correctly. In the fourth phase, MPI-based distributed communication was developed and independently validated as a replacement for the TCP socket layer, better suited to the multi-client, high-throughput communication pattern required by

multiple concurrently active Data Extraction Servers — the design decision at the heart of this report. In the fifth and final phase, the validated MPI communication layer was integrated with the processing layer and extended with the storage, graph construction, and monitoring layers, to produce the complete Hybrid Telecom Stream Processing Framework.

This incremental methodology, in which the MPI communication capability was validated in isolation before integration, is reflected directly in the organisation of the project's source code and test reports, and substantially reduced integration risk in the final system.

6.1 Phase Summary

Phase	Capability Introduced	Validated Independently	Status
1	POSIX Threads (shared buffer, worker pool)	Yes — GROUP-E_posix_threads	Superseded by hybrid integration
2	TCP Socket communication (single client)	Yes — GROUP-E_sockets	Superseded by MPI (Section 5.2.1)
3	Socket + Thread integration	Yes — GROUP-E_integration	Superseded by hybrid integration
4	MPI distributed communication, multi-DES	Yes — GROUP-E_mpi	Adopted as final communication layer
5	Full hybrid integration (MPI + processing + storage + graph + monitoring)	Yes — GROUP-F_validation	Final delivered framework

Table 6.1: Phase-Wise Development Status

Each row of Table 6.1 corresponds to an independent source directory under src/ and an independent set of test reports under tests_and_reports/, so that the correctness claims made for the final hybrid framework in Chapter 7 rest on a documented chain of independently validated components rather than on the hybrid integration alone.

6.2 Engineering Rationale and Lessons Learned per Phase

Each phase summarised in Table 6.1 contributed a specific, transferable lesson that directly informed the design of the final communication layer, described in Section 5.2, rather than being discarded once superseded.

- Phase 1 (POSIX Threads) established that a shared circular buffer, protected by a single mutex and two condition variables, could safely decouple packet arrival from packet processing; this exact synchronisation discipline was carried forward unchanged into the processing layer of the final hybrid framework, described in Section 5.4.
- Phase 2 (TCP Sockets) demonstrated a working client-server packet transport, but also exposed the connection-management burden discussed in Section 5.2.1, specifically that every new client requires explicit accept() handling and file-descriptor bookkeeping at the server; this observation directly motivated the search for a communication mechanism better suited to a growing number of concurrent clients.

- Phase 3 (Socket + Thread Integration) validated that a network communication layer and a POSIX-threaded processing layer could be composed without introducing new synchronisation hazards, giving confidence that the same composition would be safe once the socket layer was later replaced by MPI in Phase 4.
- Phase 4 (MPI) confirmed, in isolation from the processing, storage, and graph layers, that MPI_ANY_SOURCE together with a fixed server-side receive loop could accept traffic from an arbitrary, dynamically sized set of DES clients without server-side code changes, which is the single design property the entire project is built around.
- Phase 5 (Hybrid Integration) confirmed that the four previously independently validated mechanisms could be combined without any of them needing to be modified, the strongest practical evidence available that the incremental methodology described in Section 3.4 had succeeded in reducing integration risk.

Considered together, these five lessons illustrate a broader software engineering point that extends beyond this project: concurrency and distributed-communication mechanisms are difficult to debug once combined, but are comparatively tractable to validate in isolation, and a development methodology that defers integration until each mechanism is independently trusted converts a single hard integration problem into a sequence of smaller, independently verifiable ones.

7. Results and Discussion

This chapter presents the quantitative results obtained from the performance evaluation and graph validation activities, with particular attention to how the MPI communication layer performed as the number of concurrently active DES clients and the total traffic volume were scaled.

7.1 Throughput Under Increasing DES Count and Load

DES	Packets/DES	Total Pkts	Exec. Time (s)	Avg Time/Pkt (ms)	Throughput (pkts/s)
1	10	10	0.000072	0.007201	138,877.32
3	10	30	0.000240	0.008008	124,870.97
3	25	75	0.000549	0.007317	136,673.01
3	100	300	0.005964	0.019881	50,299.99
3	1000	3000	0.061620	0.020540	48,685.29

Table 7.1: Performance Evaluation Results Across DES Counts and Loads

Two distinct performance regimes are visible. For the three smallest configurations (10 to 75 total packets), throughput is very high, 124,900 to 138,900 packets per second, reflecting fixed per-execution overheads such as MPI process start-up dominating the measured time. For the two larger, sustained-load configurations (300 and 3,000 packets, both generated across three concurrent DES clients), throughput settles to a stable 48,700 to 50,300 packets per second, differing by less than four percent despite a tenfold increase in workload — strong evidence that the multi-DES MPI communication layer sustains a stable throughput without degradation as load increases a further order of magnitude.

Figure 7.1: MPI Throughput Across DES Counts and Loads

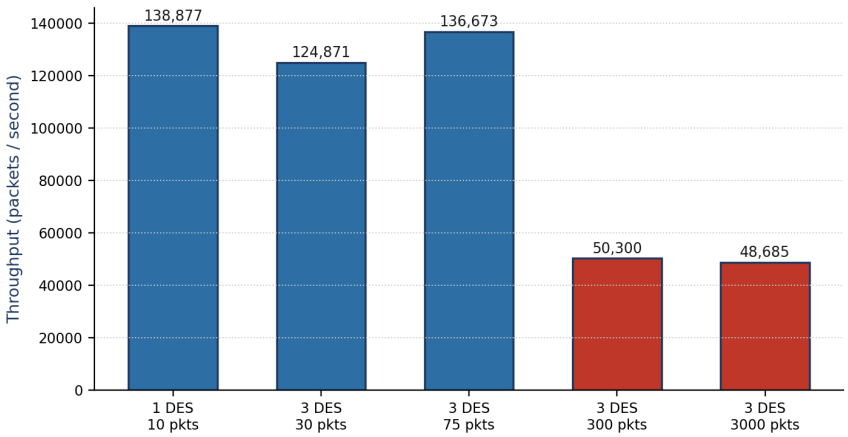

Figure 7.1: MPI Throughput Across DES Counts and Loads

7.2 Latency

The average per-packet processing time, ranging from approximately 7 microseconds under light load to approximately 20.5 microseconds under sustained load, reflects the combined cost of packet validation, buffer enqueue and dequeue operations, Berkeley DB persistence, and graph vertex and edge updates for each packet. The increase in per-packet latency observed between the lightly loaded and sustained-load regimes is consistent with increased contention for the shared circular buffer's mutex as three DES clients deliver packets concurrently, and with the increasing cost of the graph construction layer's edge lookup as the number of distinct source-destination pairs grows with sustained multi-DES traffic.

7.3 Multi-DES Correctness and Queue Behaviour

Across every test configuration, from a single DES client to three concurrently active DES clients, the number of packets received over MPI was equal to the number processed, enqueued equal to dequeued, and stored equal to the total volume generated by all DES clients combined, with no instance of packet loss, deadlock, or race condition observed. This confirms that receive_packet() correctly and fairly services multiple simultaneously connected DES clients without favouring any one client or dropping packets under contention.

7.4 Graph Construction Results

Test Configuration	Vertices	Edges	Comm. Volume	Top Link	Most Active Src → Dest
Functional (1 × 10)	10	9	10	Tower-4 → Tower-7 (2 pkts)	Tower-4 → Tower-7
Multi-DES Functional (3 × 10)	9	9	30	Tower-4 → Tower-7 (6 pkts)	Tower-4 → Tower-7
Integration (3 × 25)	10	23	75	Tower-3 → Tower-1 (6 pkts)	Tower-3 → Tower-6

Test Configuration	Vertices	Edges	Comm. Volume	Top Link	Most Active Src → Dest
Medium Load (3 × 100)	10	67	300	Tower-2 ⟶ Tower-1 (18 pkts)	Tower-3 ⟶ Tower-1
Stress (3 × 1000)	10	100	3000	Tower-4 ⟶ Tower-8 (57 pkts)	Tower-10 ⟶ Tower-8

Table 7.2: Graph Construction Validation Across Test Configurations

The vertex count remains bounded at nine or ten across all configurations, confirming that add_vertex() correctly avoids duplicate vertices, while the edge count grows from nine at a communication volume of ten packets to one hundred at three thousand packets, showing that increasing multi-DES traffic volume produces an increasingly complete picture of pairwise tower communication.

7.5 Scalability and Current Limitations

Considered together, the throughput and graph results demonstrate that the framework scales gracefully across a three-hundred-fold increase in total packet volume and a threefold increase in concurrently active DES clients, with no degradation in correctness observed at any scale tested. All testing was, however, conducted on a single physical node with co-located client and server processes, so the reported figures do not yet reflect the network latency a genuinely multi-node MPI deployment would introduce; the framework does not yet incorporate fault-tolerance mechanisms for DES client or server failure, both discussed further in Chapter 10.

8. Testing and Validation

This chapter describes the testing methodology applied to the Hybrid Telecom Stream Processing Framework and summarises the validation evidence supporting the correctness claims made in Chapter 7, with particular attention to how the MPI communication layer was verified as the number of concurrently active DES clients and the total traffic volume were scaled.

8.1 Testing Methodology and Test Configurations

Testing was conducted at two levels. At the module level, each of the four independently developed phases described in Chapter 6 — POSIX Threads, TCP sockets, socket-thread integration, and MPI communication — was independently tested and validated prior to integration, with the corresponding phase-wise test reports preserved in the GROUP-E subdirectories of tests_and_reports/. At the system level, following integration of all modules into the final hybrid framework, a comprehensive final testing programme, preserved as the GROUP-F validation report, covered functional validation, system-component validation, and performance and graph validation, using five test configurations of increasing scale, summarised in Table 8.1.

Test Case	DES	Packets/DES	Total Packets	Status
Functional Test	1	10	10	PASS

Test Case	DES	Packets/DES	Total Packets	Status
Multi-DES Functional Test	3	10	30	PASS
Integration Test	3	25	75	PASS
Medium Load Test	3	100	300	PASS
Stress Test	3	1000	3000	PASS

Table 8.1: Functional and Scalability Test Configurations

As Table 8.1 shows, the five configurations progressively increased both the number of concurrently active DES clients, from one to three, and the volume of packets generated per DES, from ten to one thousand, culminating in a stress test generating three thousand packets in total. Every configuration completed successfully, with no crashes, deadlocks, packet loss, or communication failures observed at any point.

8.2 System Component Validation

In addition to the end-to-end functional tests, every individual system component was independently validated during each of the five test executions, summarised in Table 8.2.

Component	Validation Result
MPI Communication	PASS
Packet Reception	PASS
Shared Circular Buffer	PASS
Producer-Consumer Synchronisation	PASS
Worker Thread Processing	PASS
Berkeley DB Storage	PASS
Telecom Graph Construction	PASS
Graph Analytics	PASS
Monitoring Dashboard	PASS

Table 8.2: System Component Validation Summary

For every workload tested, four internal consistency invariants were verified directly from the monitoring dashboard's reported statistics: packets received equal to packets processed; packets enqueued into the shared circular buffer equal to packets dequeued; packets stored in Berkeley DB equal to the total packets generated; and total communication volume recorded by the graph construction layer equal to the total packets generated. The simultaneous satisfaction of all four invariants, across all five configurations, demonstrates complete end-to-end correctness: no packet was lost, duplicated, or left unprocessed at any stage of the pipeline described in Section 5.3.

8.3 Communication, Processing, and Storage Testing

Communication testing verified that every packet transmitted by a DES client was correctly received by the server across all five configurations, including the multi-DES configurations in which three client processes transmitted concurrently, directly validating the many-to-one MPI pattern described in Section 5.2 and confirming that the communication layer scales correctly to multiple concurrently active clients without message loss or misattribution. Processing testing verified that the shared circular buffer and worker thread pool correctly processed every enqueued packet, with dequeue counts equal to enqueue counts in every configuration, confirming the correctness of the mutex and condition-variable synchronisation discipline described in Section 5.4 under the increased contention of the medium-load and stress configurations. Database testing verified that every processed packet was correctly persisted to Berkeley DB, with stored counts equal to generated counts in every configuration.

8.4 Integration and Scalability Testing

Integration testing, corresponding to the Integration Test row of Table 8.1 (three DES clients, twenty-five packets each, seventy-five packets total), verified correct end-to-end operation across all five modules simultaneously and passed without incident. Scalability testing examined behaviour as both the DES count and per-server packet volume were increased, culminating in the stress configuration of three DES clients each generating one thousand packets. The framework completed this stress test successfully, with all correctness invariants preserved and throughput remaining stable in the range reported in Section 7.1.

8.5 Testing Limitations

All test configurations were executed on a single physical node, with DES client and server processes co-located rather than distributed across a networked cluster; the measured performance figures therefore reflect local inter-process MPI communication rather than communication over a wide-area or local-area network, and may not directly generalise to a genuinely distributed, multi-node deployment, discussed further as future work in Chapter 10. The number of distinct telecom towers used during traffic generation was also comparatively small, sufficient to demonstrate correct graph construction and meaningful top-link identification, but not to exercise the graph construction layer's behaviour under the much larger vertex and edge counts a production-scale deployment might present.

8.6 Software Engineering Value of the Testing Discipline

Beyond establishing the specific correctness and performance results reported in Chapters 7 and 8, the testing discipline applied throughout this project served a broader software engineering purpose consistent with the incremental methodology described in Section 3.4 and Section 6.2. Because a dedicated validation report was produced and preserved for every phase, described in Section 8.1, a regression introduced during hybrid integration could, in principle, be localised by comparing the integrated system's behaviour against the previously validated behaviour of the specific module suspected of having changed, rather than by re-deriving correctness for the whole system from first principles. This property was exercised in practice during the transition from the phase-four MPI-only implementation to the phase-five hybrid integration, described in Section 6.1, where the phase-four MPI validation results

provided a known-good baseline against which the newly integrated processing, storage, and graph layers could be checked for interference with the already-validated communication behaviour.

9. Software Developed

This chapter describes precisely what software was developed during the internship and how it can be built and used. The deliverable is the complete telecom-stream-processing-framework repository, comprising five standalone phase implementations and the final integrated hybrid framework, all implemented in C for Ubuntu Linux.

9.1 What Was Developed

- An MPI-based client-server communication module supporting an arbitrary number of concurrently connected Data Extraction Server clients (src/mpi and src/hybrid/mpi) — the primary deliverable of this project.
- A TelecomPacket data module defining the packet format and providing initialisation, validation, clearing, and printing functions, shared across every phase.
- A traffic-generation module that synthesises realistic telecom packets in the absence of a live network feed, used by every DES client during testing.
- A POSIX-threaded processing module implementing a shared circular buffer and a configurable worker thread pool (src/pthreads and src/hybrid/processing).
- A Berkeley DB storage module for persisting processed packets (src/hybrid/storage).
- A graph construction module that models tower-to-tower communication as a weighted graph and reports the most active communication link (src/hybrid/graph).
- A monitoring module that aggregates and displays live statistics from every other module (src/hybrid/monitoring).
- Standalone, independently buildable implementations of each development phase — pthreads-only, TCP-sockets-only, socket-thread integration, and MPI-only — preserved for reference alongside the final hybrid/ implementation.
- The academic submission deliverables for the internship — this thesis report, the accompanying presentation slides, and PDF renderings of both — maintained under docs/submissions/ alongside the technical documentation described in Section 3.3.

9.2 How to Build It

The framework requires GCC, GNU Make, OpenMPI, and the Berkeley DB development libraries, all available through the standard Ubuntu package repositories.

Step	Command
Update package index	sudo apt update
Install build essentials	sudo apt install build-essential
Install OpenMPI	sudo apt install openmpi-bin libopenmpi-dev
Install Berkeley DB	sudo apt install libdb-dev

Step	Command
Build MPI phase only	cd src/mpi && make
Build hybrid framework	cd src/hybrid && make

Table 9.1: Build Instructions

9.3 How to Run and Use It

The hybrid framework is launched using mpirun, with the process count controlling the number of concurrently active MPI processes, one of which acts as the server and the remainder as DES clients:

```
mpirun -np 4 ./hybrid
```

The command above launches one server process and three DES client processes, matching the multi-DES configuration evaluated in Chapter 7. Increasing the -np argument increases the number of concurrently connected DES clients without any change to the source code, directly exercising the multi-DES design described in Section 5.2. While running, monitoring_print_dashboard() periodically prints packet counts, queue occupancy, throughput, and graph statistics to the terminal, providing a live view of the system's behaviour; on completion, processed packets are persisted in telecom.db and can be retrieved individually using db_get_packet().

9.4 Consolidated Public API Reference

Module	Function	Description
Communication	initialize_server() / initialize_client()	Set up the MPI server and DES client roles respectively.
Communication	send_packet() / receive_packet()	Transmit and receive a single TelecomPacket over MPI.
Telecom Packet	initialize_packet(), clear_packet(), validate_packet(), print_packet()	Create, reset, validate, and display a packet.
Traffic Generation	generate_packet(), generate_source(), generate_destination(), generate_traffic_type(), generate_timestamp(), generate_payload()	Synthesise realistic telecom packets for testing.
Processing	buffer_init(), enqueue(), dequeue(), process_telecom_packet()	Shared-buffer producer-consumer packet processing.
Storage	db_initialize(), db_store_packet(), db_get_packet(), db_close()	Persist and retrieve packets via Berkeley DB.
Graph Construction	initialize_graph(), add_vertex(), add_edge(), print_graph_statistics(), get_top_communication_link()	Build and query the tower communication graph.
Monitoring	monitoring_initialize(), monitoring_get_statistics(), monitoring_print_dashboard()	Aggregate and display runtime statistics.

Table 9.2: Consolidated Public API Reference

9.5 Extending the Framework to More DES Clients

Because the communication layer's design places no upper bound on the number of DES clients it can service, a user wishing to evaluate the framework under a larger multi-DES workload than the three-client configuration reported in Chapter 7 need only increase the -np argument passed to mpirun, for example mpirun -np 6 ./hybrid to launch five DES clients against one server, and, optionally, adjust the packets-per-DES parameter exposed by the traffic-generation module described in Section 5.3 to control the resulting load. No source code change is required for either adjustment, which is, in itself, the practical demonstration of the multi-DES design goal stated in Chapter 1.

10. Conclusions and Future Directions

10.1 Conclusions

This report has presented the design, implementation, and evaluation of the Hybrid Telecom Stream Processing Framework, developed during a summer internship at MPPLAB, C-DAC Pune, under the guidance of Dr. V. C. V. Rao. The project's central achievement is an MPI-based communication layer that supports an arbitrary number of concurrently active Data Extraction Servers without modification to the server's core logic, validated across configurations ranging from one to three concurrent DES clients and from ten to three thousand packets.

Across every test configuration, the framework demonstrated complete correctness, with no packet loss, deadlock, or synchronisation failure observed, and sustained a stable throughput of approximately 48,000 to 50,000 packets per second under load. The supporting processing, storage, graph construction, and monitoring layers built around this communication core were likewise shown to correctly and consistently handle the traffic delivered by the MPI layer, with the graph construction module correctly identifying the most active communication links across every configuration tested.

The incremental development methodology, in which POSIX Threads, TCP sockets, socket-thread integration, and MPI were each independently validated before combination, substantially reduced integration risk and is reflected throughout the organisation of the project repository.

From an internship-learning perspective, the project provided direct, hands-on experience with the practical differences between socket-based and message-passing-based distributed communication models, with the disciplines of thread synchronisation using mutexes and condition variables, and with the process of validating a distributed system incrementally, one capability at a time, rather than attempting to debug a fully integrated system all at once. These are transferable skills directly applicable to the broader parallel and distributed computing work carried out at MPPLAB.

10.2 Future Directions

- Distributed cluster deployment: extending the present single-node deployment to a genuinely multi-node MPI configuration, requiring no change to the core communication logic, only to the deployment configuration and MPI host file.
- Dynamic load balancing of worker threads, building on the statistics already reported by the monitoring layer.
- Fault tolerance and recovery for DES client, server, or storage-layer failure during execution, including retry logic for failed MPI transmissions.
- Cloud-native deployment and containerisation using Docker, particularly for the multi-node MPI configuration.
- Advanced telecom traffic analytics connecting the communication graph to the broader MPPLAB tele-traffic mathematical modelling work.
- Web-based visualisation of the communication graph, rendering the vertices and weighted edges maintained by the graph construction layer interactively.

Collectively, these directions chart a path from the present internship-scope proof-of-concept towards a production-capable, MPI-based telecom data extraction and processing platform.

10.3 Reflections on Software Engineering Practice

Considered as a piece of software engineering rather than solely as a piece of parallel and distributed systems engineering, the project's principal lesson is that the two are inseparable in practice: the correctness properties claimed in Chapter 7 and validated in Chapter 8 rest as much on the modular architecture, narrow interfaces, and incremental validation discipline described in Section 5.7 and Section 6.2 as they do on any individual technical choice of MPI over TCP sockets. A technically sound communication mechanism, developed and integrated without regard for these practices, would have been considerably harder to debug, extend to multiple DES clients, and hand off for future maintenance than the mechanism actually delivered.

This has a direct, practical implication for the future directions listed above: each of them, from distributed cluster deployment to graph visualisation, is more tractable precisely because the present framework was built with clearly separated layers, documented interfaces, and a preserved record of independently validated phases, described in Section 3.6 and Chapter 6, rather than as a single undifferentiated program. Future MPPLAB interns or staff extending this work inherit not only a working communication layer but a documented methodology for extending it safely.